Практика по дисциплине

Теория принятия решений

Вводное занятие 1

Пономарёв Андрей Васильевич

Организационное

- Много информации, имеющей отношение к задачам:
  - **https://a-v-ponomarev.github.io/** (далее просто Сайт)
- Три задачи – как правило, две ЛП, одна ДП (есть исключения):
  - Разработать математическую модель
  - Найти решение задачи оптимизации
  - Ответить на вопросы по заданию (исследование полученного решения)
- Варианты:
  - Сами тексты уже на Сайте
  - Распределение вариантов - ответом на письмо от старосты

Организационное. Отчетность

- Экзамен –> Мустафин Николай Алексеевич
- Практические задания:
  - Влияет на экзамен
  - Отчет:
    - Содержание: формализация, решение, интерпретация, ответы на доп. вопросы (см. Сайт)
    - Оформление: приличное, с достаточным количеством пояснений и визуализаций
    - Формат: блокнот Jupyter или PDF в соответствии с шаблоном на Сайте
  - Индивидуальная защита (очно):
    - Могу попросить изменить какое-нибудь условие, что-то добавить и т.п.
    - Перед защитой прислать электронную версию отчета (**тема письма: «ТПР: XXX», где XXX – номер варианта**)
    - Можно сдавать задачи по мере готовности

Компонент оценки за практику	Максимальный балл
Защита задачи (*3)	30
Небрежное оформление	-5

Рекомендуемая оценка	Баллы
Отлично	80 – 90
Хорошо	50 – 79
Удовлетворительно	30 – 49

Организационное. Фазы

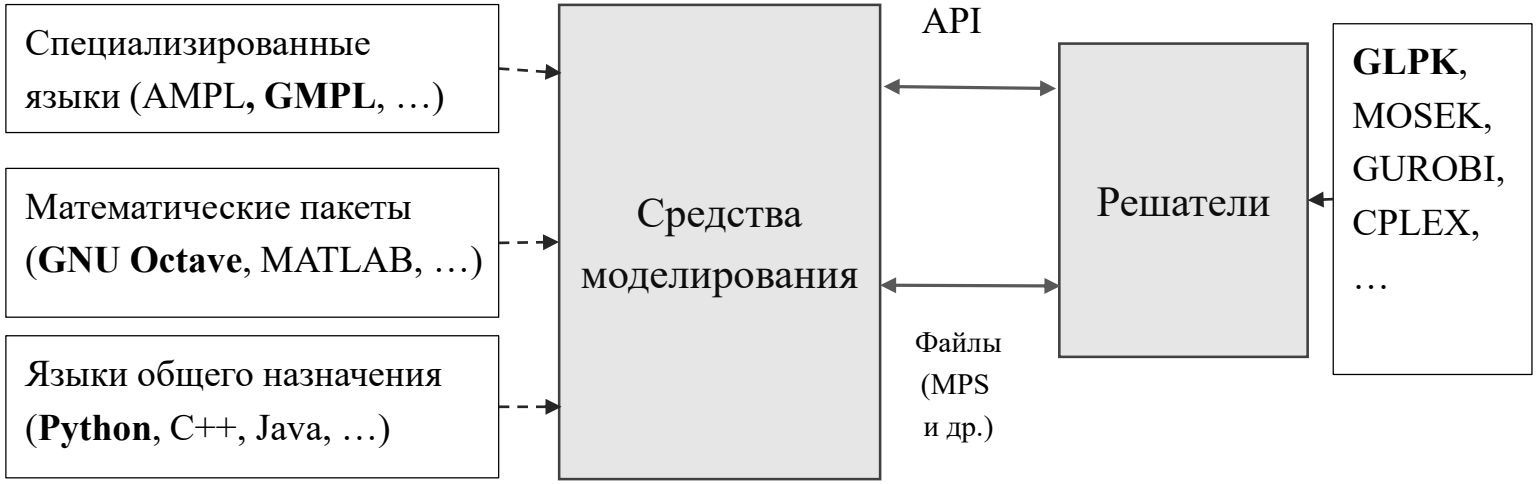
- Фазы нашего взаимодействия:
  1. Введение [*примерно до середины апреля*]
     - разбор у доски отдельных задач и «сюжетов», в них используемых (ЛП, ДП, марковские процессы принятия решений, диаграммы влияния)
     - индивидуальные консультации по задачам в оставшееся время
  2. Консультации + прием работ [*с середины апреля*]
  3. Прием работ (май)

Программное обеспечение

- Инструменты:
  - Рекомендуемый инструмент: Python (Jupyter, CVXOPT/PuLP, seaborn, pandas etc.)
  - Возможно, но не рекомендуется (legacy mode):
    - Для задачи динамического программирования - Octave/C++/Python/Java
    - Для линейного программирования – Octave (glpk) или GLPK (через GUSEK)
- Почему это так?
  - Вы решали задачи линейного программирования, транспортные задачи, "руками" в соответствующих курсах
  - В рамках этого курса будет полезно познакомиться с тем, что представляет собой мир **ПО для оптимизации** и "пощупать" его руками
  - Python. Популярная инфраструктура для обработки и *анализа данных*
    - + язык достаточно широкого назначения (чем выгодно отличается от Octave/R и пр.)

Программное обеспечение. Пояснения

- В мире оптимизации существует два класса приложений:
  - 1) **программные средства для моделирования** - с их помощью вы на понятном, удобном синтаксисе записываете математическую модель вашей задачи.
  - 2) **солверы**. Они либо встраиваемые (линкуются как библиотеки), либо считывают файлы какого-либо из проприетарных или распространенных форматов (mps).



План

- Постановка задачи линейного программирования (ЗЛП) (почти как в индивидуальных заданиях)
- Решение ЗЛП в Python
- Решение ЗЛП в GNU Octave (*очень бегло*)
- Решение ЗЛП в GLPK/MathProg (*очень бегло*)

Постановка задачи линейного программирования (пример)

Условие

- Рацион стада крупного рогатого скота включает **пищевые продукты A, B, C, D, E** (витамины/микроэлементы/питательные вещества).
- В сутки одно животное должно съедать **не менее A1 [кг] продукта A, B1 [кг] продукта B, C1 [кг] продукта C, D1 [кг] продукта D, E1 [кг] продукта E (суточные нормы).**
- В чистом виде указанные продукты не производятся, они содержатся в кормовых культурах **К-1, К-2, К-3, К-4**. Известно **долевое содержание** каждого из продуктов в каждой из культур.
- Известен **запас каждой из кормовых культур (в т)**.
- Определить **максимальное поголовье скота**, которое можно содержать **в течение года**, используя имеющийся запас культур.

Решение

- *Параметры*
  - Будем использовать i для индексирования продуктов, j – для индексирования культур.
  - Долевое содержание (r_{ij}), $i \in \{1..5\}$, $j \in \{1..4\}$.
  - Суточные нормы продуктов d_i, $i \in \{1..5\}$ [кг].
  - Запас культур s_j [т].
- Переменные
  - n – количество животных;
  - $x_j \in \{1..4\}$ – количество культуры j-того типа, которую мы используем в кормовом плане [т].
- Тогда

  (ЦФ) n -> max

  $x_1 r_{11} + x_2 r_{12} + x_3 r_{13} + x_4 r_{14} >= 0{,}365\ d_1\ n$ - слева количество первого продукта

  …

  $x_1 <= s_1$ (согласовать единицы измерения!)

Линейные задачи. Таблица

	X1	**X2**	**X3**	**X4**	**n**	**Неравенство**	**Правая часть**
f	0	0	0	0	1	-	max
y1	r_{11}	r_{12}	r_{13}	r_{14}	-0,365*d_1	>=	0
…	…	…	…	…	…	…	…
y6	1	0	0	0	0	<=	s_1
…	…	…	…	…	…	<=	…

Решение задачи ЛП в Python

Конфигурирование среды

1. Установить менеджер пакетов, зависимостей и окружений – Conda (Miniconda)
   - Особенно важно для Windows
   - Вариант: весь дистрибутив Anaconda (но это overkill)
2. Сконфигурировать окружение:

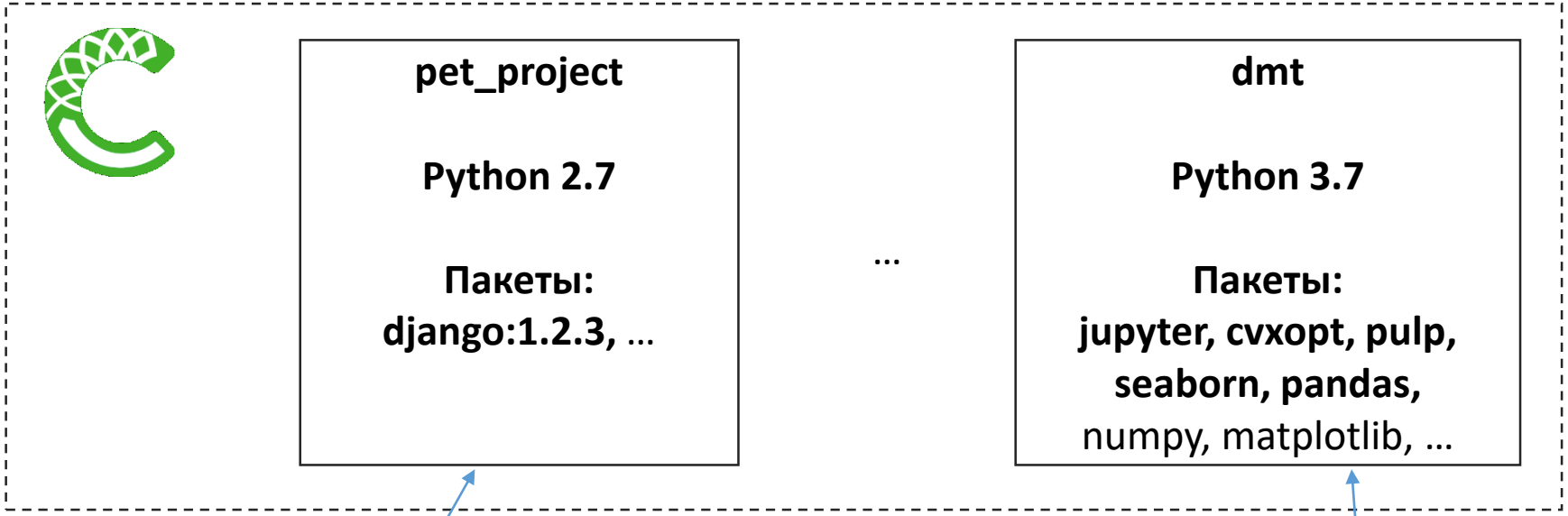
```
conda create --name dmt python=3.11
conda activate dmt
conda install jupyter cvxopt pulp seaborn pandas
jupyter notebook
```

или

создать проект в своей любимой IDE для Python (Pycharm?) указав, что он должен выполняться в соответствующем окружении

Больше информации на https://conda.io/

Менеджер пакетов, зависимостей и окружений Conda



Anaconda Prompt

```
(base) C:\> activate dmt
(dmt) C:\> jupyter notebook
```

Больше информации на https://conda.io/

Docker

```
$ docker pull jupyter/minimal-notebook
$ docker run -it --rm -p 8888:8888 jupyter/minimal-notebook
```

Внутри контейнера:

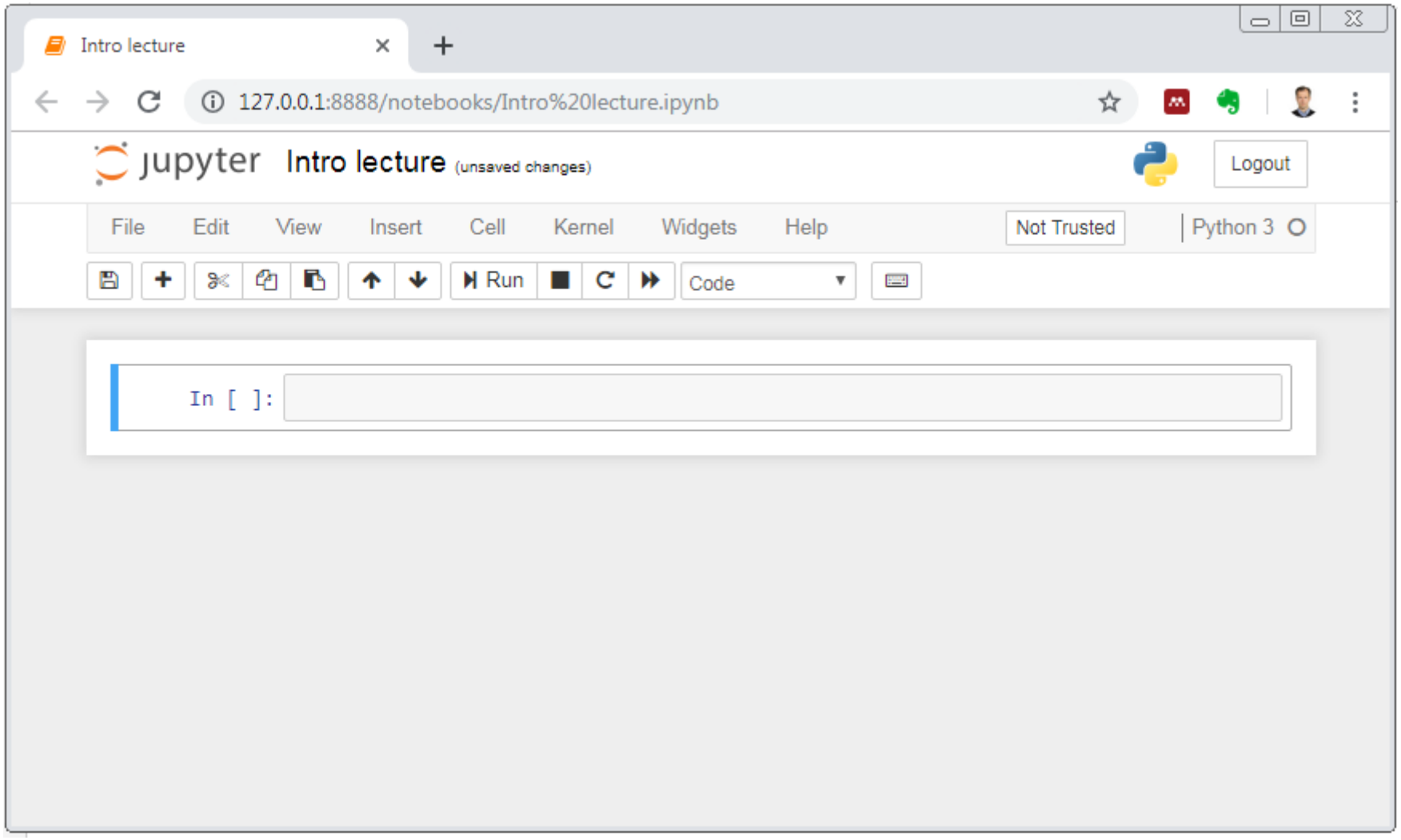
```
$ conda install cvxopt pulp seaborn pandas
```

Google Colaboratory

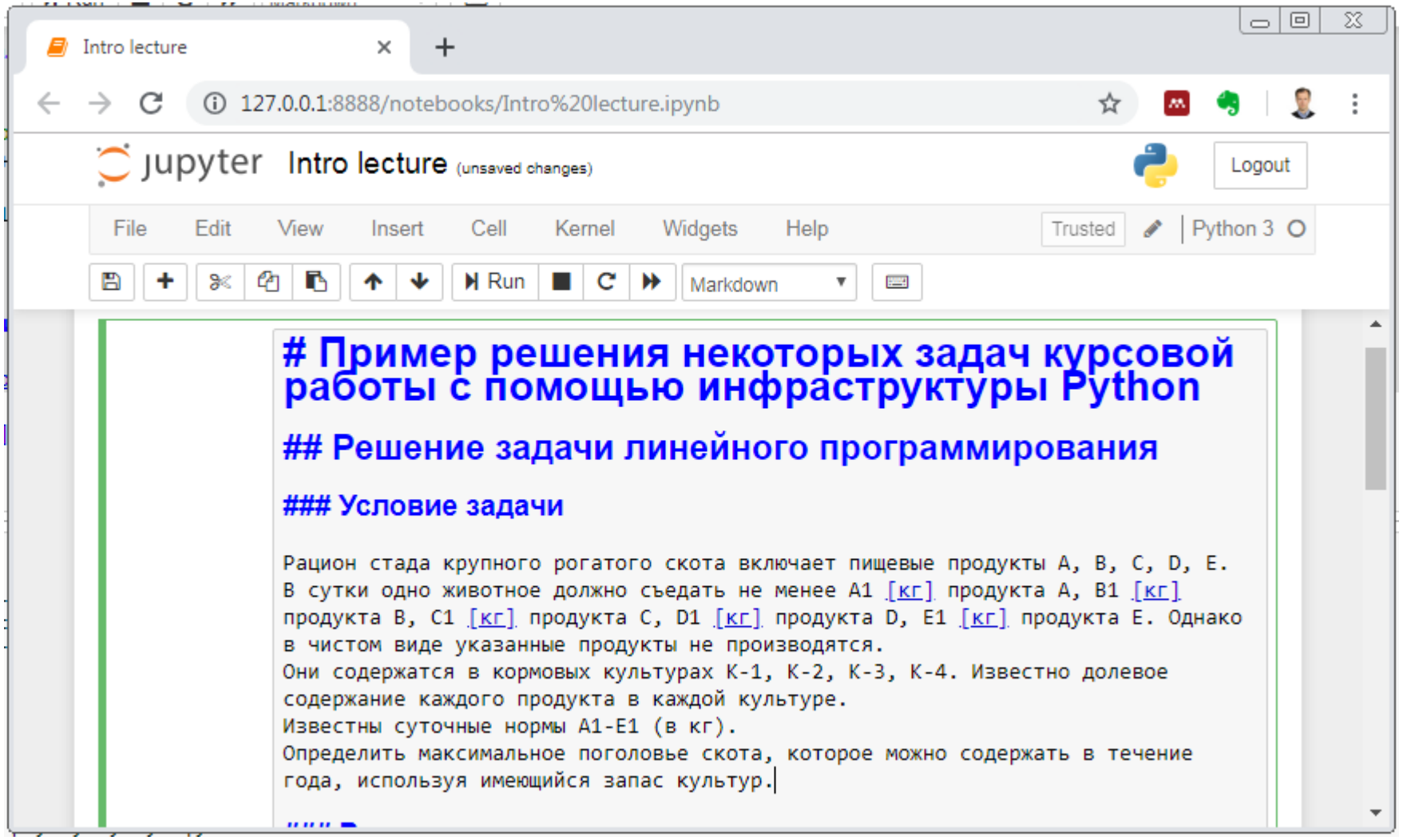
Некоторые библиотеки Python для решения задач ЛП

Библиотека	Матричный интерфейс	Моделирование	MILP	Анализ чувствительности	Пакет conda	Внешние солверы
CVXOPT	**+**	**+/-**	**+**	**+**	**+**	**+**
PuLP	**-**	**+**	**+**	**-**	**+**	**+**
scipy.optimize.linprog	+	-	-	-	+	-
Pyomo	-	+	+	?	+	?

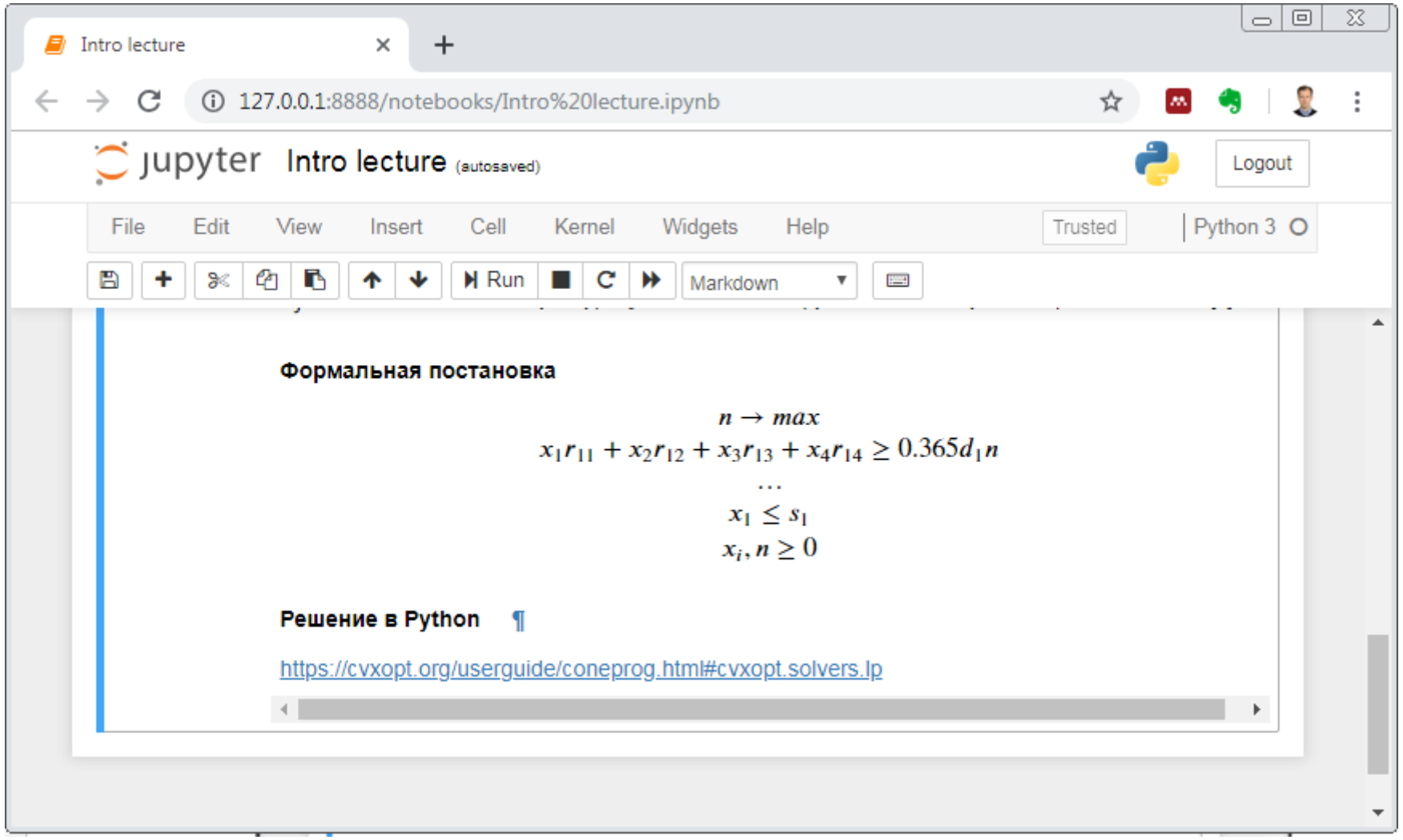
Использование Jupyter



Использование Jupyter



Использование Jupyter



Использование CVXOPT

`cvxopt.solvers.lp`(*c*, *G*, *h*[, *A*, *b*[, *solver*[, *primalstart*[, *dualstart*]]]])

$$\begin{array}{ll} \text{minimize} & c^T x \\ \text{subject to} & Gx + s = h \\ & Ax = b \\ & s \succeq 0 \end{array}$$

$$\begin{array}{ll} \text{maximize} & -h^T z - b^T y \\ \text{subject to} & G^T z + A^T y + c = 0 \\ & z \succeq 0. \end{array}$$

Таким образом:

c – коэффициенты переменных в целевой функции;
G – коэффициенты переменных в ограничениях-неравенствах (<=);
h – правые части ограничений-неравенств (<=);
A – коэффициенты переменных в ограничениях-равенствах;
b – правые части ограничений-равенств.

Использование CVXOPT: заполнение матриц

	x1	x2	x3	x4	n	Неравенство	Пр. часть
c	0	0	0	0	1	-	max (-1)
y1	r_{11}	r_{12}	r_{13}	R_{14}	$-0{,}365*d_1$	>= (-1)	0
…	…	…	…	…	…	…	…
y6	1	0	0	0	0	<=	s_1
…	…	…	…	…	…	…	…
y11	1	0	0	0	0	>= (-1)	0
…	…	…	…	…	…	…	…

Использование CVXOPT: заполнение матриц

c – коэффициенты переменных в целевой функции

	x1	x2	x3	x4	n	Неравенство	Пр. часть
c	0	0	0	0	1	-	max (-1)
y1	r_{11}	r_{12}	r_{13}	r_{14}	$-0{,}365*d_1$	>= (-1)	0
…	…	…	…	…	…	…	…
y6	1	0	0	0	0	<=	s_1
…	…	…	…	…	…	…	…
y11	1	0	0	0	0	>= (-1)	0
…	…	…	…	…	…	…	…

```
from cvxopt import matrix, solvers

c = matrix([0, 0, 0, -1], tc='d')
```

Использование CVXOPT: заполнение матриц

G – коэффициенты переменных в ограничениях-неравенствах

	x1	x2	x3	x4	n	Неравенство	Пр. часть
c	0	0	0	0	1	-	max (-1)
y1	r_{11}	r_{12}	r_{13}	r_{14}	$-0{,}365*d_1$	>= (-1)	0
…	…	…	…	…	…	…	…
y6	1	0	0	0	0	<=	s_1
…	…	…	…	…	…	…	…
y11	1	0	0	0	0	>= (-1)	0
…	…	…	…	…	…	…	…

```
from cvxopt import matrix, solvers

c = matrix([0, 0, 0, -1], tc='d')

G = matrix([[-r11,-r12,-r13,-r14,0.365*d1],
            [-r21,-r22,-r23,-r24,0.365*d2],
            …
            [  1,    0,    0,    0,       0],
            [  0,    1,    0,    0,       0],
            [  0,    0,    1,    0,       0],
            [  0,    0,    0,    1,       0],
            [ -1,    0,    0,    0,       0],
            [  0,   -1,    0,    0,       0],
            [  0,    0,   -1,    0,       0],
            [  0,    0,    0,   -1,       0],
            [  0,    0,    0,    0,      -1]
           ], tc='d')
```

Использование CVXOPT: заполнение матриц

h – значения правых частей ограничений-неравенств

	x1	x2	x3	x4	n	Неравенство	Пр. часть
c	0	0	0	0	1	-	max (-1)
y1	r_{11}	r_{12}	r_{13}	r_{14}	$-0{,}365*d_1$	>= (-1)	0
…	…	…	…	…	…	…	…
y6	1	0	0	0	0	<=	s_1
…	…	…	…	…	…	…	…
y11	1	0	0	0	0	>= (-1)	0
…	…	…	…	…	…	…	…

```
from cvxopt import matrix, solvers

c = matrix([0, 0, 0, -1], tc='d')

G = matrix([[-r11,-r12,-r13,-r14,0.365*d1],
            [-r21,-r22,-r23,-r24,0.365*d2],
            …
            [  1,    0,    0,    0,        0],
            [  0,    1,    0,    0,        0],
            [  0,    0,    1,    0,        0],
            [  0,    0,    0,    1,        0],
            [ -1,    0,    0,    0,        0],
            [  0,   -1,    0,    0,        0],
            [  0,    0,   -1,    0,        0],
            [  0,    0,    0,   -1,        0]
            [  0,    0,    0,    0,       -1]
           ], tc='d')

h = matrix([0, 0, 0, 0, 0, 0,
            s1, s2, s3, s4
            0, 0, 0, 0, 0], tc='d')
```

Пример решения ЗЛП с помощью CVXOPT

```
from cvxopt import matrix, solvers

c = matrix([0, 0, 0, 0, -1], tc='d')
G = matrix([[-0.05,-0.13,    0,    0,  0.365 * 1.5],
            [-0.10,-0.20,-0.10,    0,  0.365 * 1.4],
            [-0.04,-0.06,-0.15,    0,  0.365 * 1.0],
            [   0, -0.10,-0.08,-0.15,  0.365 * 0.8],
            [-0.15,    0,-0.14,-0.10,  0.365 * 0.6],
            [1, 0, 0, 0, 0],
            [0, 1, 0, 0, 0],
            [0, 0, 1, 0, 0],
            [0, 0, 0, 1, 0],
            [-1, 0, 0, 0, 0],
            [ 0,-1, 0, 0, 0],
            [ 0, 0,-1, 0, 0],
            [ 0, 0, 0,-1, 0],
            [ 0, 0, 0, 0,-1],
           ], tc='d')
```

```
h = matrix([0, 0, 0, 0, 0, 5, 6, 7, 8,
                                  0, 0, 0, 0, 0], tc='d')

solution = solvers.lp(c, G.T, h, solver='glpk')
print('Status: ', solution['status'])
print('x = \n', solution['x'])
print('Objective: ', solution['primal objective'])
```

```
Status: optimal
x =
[ 5.00e+00]
[ 6.00e+00]
[ 8.44e-01]
[ 0.00e+00]
[ 1.88e+00]
Objective: -1.881278538812786
```

Моделирование с помощью CVXOPT (1)

```
from cvxopt.modeling import variable, op

# Определение переменных
x = variable(4, 'x')
n = variable(1, 'n')

# Определение задачи оптимизации
# (всегда минимизация)
problem = op(-x[0] - x[1],    # Ц.Ф.
             [x[0] + x[1] + x[2] + x[3] <= 4,
              x >= 0,
              n >= 0])

# Решение задачи
problem.solve(solver='glpk')
```

```
print('Status: ', problem.status)
print('x = \n', x.value)
print('Objective: ', problem.objective.value())
```

```
Status: optimal
x = [ 4.00e+00]
    [ 0.00e+00]
    [ 0.00e+00]
    [ 0.00e+00]
Objective: [-4.00e+00]
```

Моделирование с помощью CVXOPT (2)

```
from cvxopt.modeling import variable, op

# Определение переменных
x = variable(4, 'x')

# Определение задачи оптимизации
# (всегда минимизация)
problem = op(-sum(x),    # Ц.Ф.
             [x[i] <= x[i+1] - 1 for i in range(3)] + \
             [x <= 10,
              x >= 0])

# Решение задачи
problem.solve(solver='glpk')
```

```
tmp = []
for i in range(3):
    tmp.append(x[i] <= x[i+1] - 1)
```

См. также **PuLP**